02_hunyuanvideo_setup_and_first_video.ipynb ☆ Saving...

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

Share Reconnect A100 High-RAM

```python
#Verify L4 GPU
import subprocess
result = subprocess.run(['nvidia-smi'], capture_output=True, text=True)
print(result.stdout)
```

```
Thu Jul 16 19:21:22 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA A100-SXM4-80GB          Off |   00000000:00:05.0 Off |                    0 |
| N/A   36C    P0             51W /  400W |       0MiB /  81920MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```python
#Verify PyTorch on L4
import torch
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Total GPU memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
```

```
PyTorch version: 2.11.0+cu128
CUDA available: True
GPU: NVIDIA A100-SXM4-80GB
Total GPU memory: 85.09 GB
```

```python
#Mounting Google Drive
from google.colab import drive
drive.mount('/content/drive')

DRIVE_PROJECT = '/content/drive/MyDrive/msc_deepfake_project'
print(f"Project folder: {DRIVE_PROJECT}")
```

```
Mounted at /content/drive
Project folder: /content/drive/MyDrive/msc_deepfake_project
```

```python
#Install HunyuanVideo dependencies
!pip install -q --upgrade diffusers transformers accelerate imageio imageio-ffmpeg sentencepiece
!pip install -q einops safetensors

# Verify
import diffusers, transformers, torch
print(f"diffusers: {diffusers.__version__}")
print(f"transformers: {transformers.__version__}")
print(f"torch: {torch.__version__}")
print("\nAll HunyuanVideo dependencies installed.")
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.6/11.6 MB 125.8 MB/s eta 0:00:00
diffusers: 0.39.0
transformers: 5.14.1
torch: 2.11.0+cu128

All HunyuanVideo dependencies installed.
```

```python
#Loading HunyuanVideo on A100 (no CPU offload needed)
from diffusers import HunyuanVideoPipeline, HunyuanVideoTransformer3DModel
import torch
import gc

gc.collect()
torch.cuda.empty_cache()

print("Loading HunyuanVideo model on A100...\n")

transformer = HunyuanVideoTransformer3DModel.from_pretrained(
    "hunyuanvideo-community/HunyuanVideo",
    subfolder="transformer",
    torch_dtype=torch.bfloat16
)

pipe = HunyuanVideoPipeline.from_pretrained(
    "hunyuanvideo-community/HunyuanVideo",
    transformer=transformer,
    torch_dtype=torch.bfloat16
)

# A100 has plenty of memory — move everything to GPU, no offload needed
pipe.to("cuda")

# Keep VAE optimisations for stability (they're safe on A100 too)
pipe.vae.enable_tiling()
pipe.vae.enable_slicing()

print("\nHunyuanVideo model loaded on A100 (no CPU offload).")
print(f"GPU memory used: {torch.cuda.memory_allocated()/1e9:.2f} GB")
print(f"GPU memory available: {torch.cuda.get_device_properties(0).total_memory/1e9:.2f} GB")
```

```
Loading HunyuanVideo model on A100...

Download complete: :    0.00B

Reconstruction complete:    0.00B /  0.00B

Fetching 6 files: 100%  6/6 [00:00<00:00, 612.99it/s]

Loading checkpoint shards: 100%  6/6 [00:00<00:00, 43.46it/s]

Loading pipeline components...: 100%  7/7 [00:05<00:00,  1.03it/s]

Loading weights: 100%  196/196 [00:00<00:00, 2495.21it/s]

Loading weights: 100%  290/290 [00:02<00:00, 103.56it/s]

HunyuanVideo model loaded on A100 (no CPU offload).
GPU memory used: 63.71 GB
GPU memory available: 85.09 GB
```

```python
#Generating on A100
import torch
from diffusers.utils import export_to_video
from datetime import datetime

prompt = "A woman with long brown hair smiling gently at the camera in a sunlit park, natural lighting, soft breeze moving her hair, cinematic quality"

generator = torch.Generator(device="cuda").manual_seed(42)

print(f"Prompt: {prompt}\n")
print("Generating video on A100...")

start_time = datetime.now()

video_frames = pipe(
```

```python
video_frames = pipe(
    prompt=prompt,
    height=480,
    width=704,
    num_frames=73,
    num_inference_steps=30,
    guidance_scale=6.0,
    generator=generator,
).frames[0]

end_time = datetime.now()
duration = (end_time - start_time).total_seconds()

print(f"\nGeneration complete in {duration:.1f} seconds ({duration/60:.1f} minutes)")
print(f"Number of frames: {len(video_frames)}")
```

```
Prompt: A woman with long brown hair smiling gently at the camera in a sunlit park, natural lighting, soft breeze moving her hair, cinematic quality

Generating video on A100...
100% 30/30 [03:56<00:00,  7.95s/it]

Generation complete in 249.4 seconds (4.2 minutes)
Number of frames: 73
```

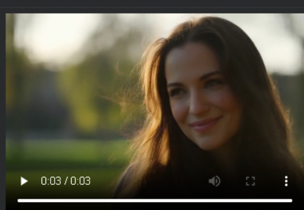
[ ]

```python
# Saving HunyuanVideo output to Drive
from datetime import datetime
import json

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
video_id = f"hunyuan_001_{timestamp}"

video_path = f'{DRIVE_PROJECT}/videos/generated/hunyuan/{video_id}.mp4'
metadata_path = f'{DRIVE_PROJECT}/metadata/{video_id}.json'

# Export video
export_to_video(video_frames, video_path, fps=24)

# Save metadata (mirrors LTX metadata format for consistency)
metadata = {
    "video_id": video_id,
    "generator": "HunyuanVideo",
    "generator_version": "hunyuanvideo-community/HunyuanVideo",
    "prompt": prompt,
    "negative_prompt": None,
    "seed": 42,
    "width": 704,
    "height": 480,
    "num_frames": 73,
    "fps": 24,
    "num_inference_steps": 30,
    "guidance_scale": 6.0,
    "generation_time_seconds": duration,
    "generation_datetime": timestamp,
    "gpu_used": "L4",
    "optimizations": ["cpu_offload", "vae_tiling", "vae_slicing"]
}

with open(metadata_path, 'w') as f:
    json.dump(metadata, f, indent=2)

print(f"Video saved: {video_path}")
print(f"Metadata saved: {metadata_path}")
```

```
Video saved: /content/drive/MyDrive/msc_deepfake_project/videos/generated/hunyuan/hunyuan_001_20260716_193318.mp4
Metadata saved: /content/drive/MyDrive/msc_deepfake_project/metadata/hunyuan_001_20260716_193318.json
```

[ ]

```python
# Previewing the video inline
from IPython.display import Video
Video(video_path, embed=True, width=400)
```



[ ]

```python
#Downloading to local machine
from google.colab import files
files.download(video_path)
```

...